SKILLNOVA ACADEMY

Comprehensive Codebase Audit & Professional Roadmap

Architectural Blueprint for Transforming the BS CS Final Year Project into an Enterprise-Grade Commercial LMS

Project: SkillNova Academy LMS (Full-Stack Next.js 16)	**Target Stack:** Next.js 16 • PostgreSQL • Prisma • AWS S3 • Stripe
Author / Lead: Murtaza Wazir (BS CS FYP)	**Audit Date:** October 17, 2026
Evaluation Scope: Security, DB, LMS Classroom, Real-time, DevOps	**Target State:** Production-Ready Commercial Cloud Platform

1. Executive Summary & Health Scorecard

A rigorous codebase analysis was conducted across all subsystems of the **SkillNova-FYP** platform. The project demonstrates impressive aesthetic quality, modern Next.js 16 Turbopack architecture, modular server actions, and clean role-based dashboards (Student, Trainer, Admin). However, to transition from an academic prototype into a **bulletproof, enterprise-ready commercial LMS**, critical technical debt must be addressed in authentication security, database concurrency, media delivery, financial transactional integrity, and automated test coverage.

Subsystem	Current State	Health	Critical Deficiencies & Production Risks
Authentication & RBAC	Unsigned Base64 cookie, hardcoded backdoor passwords	HIGH RISK	Cookie tampering allows privilege escalation to ADMIN without secret validation. Backdoor credentials exist in production paths.
Database & Persistence	Single-file SQLite (prisma/dev.db), no migrations	MEDIUM	File-lock write contention under concurrent requests. Ephemeral cloud deployments (Vercel/Docker) lose database on container recycling.
Media & Storage	Hardcoded YouTube iframes, Unsplash image URLs	MEDIUM	No private video DRM, no student assignment file uploads, no S3/Cloudinary CDN integration for custom content hosting.
Financial & E-Commerce	Zero-dollar immediate enrollment bypass	CRITICAL	Students can enroll in paid courses without checkout or payment verification. No Stripe/PayFast gateway, order ledger, or tax invoices.
Route Protection	Ad-hoc page-level redirection, no middleware.ts	MEDIUM	Missing centralized Next.js middleware. Static assets or future API endpoints lack uniform token inspection and security headers.

Testing & DevOps	No unit tests, no E2E tests, no CI/CD pipeline	**HIGH RISK**	Zero automated test coverage. Regressions in quiz grading, progress calculation, or auth can go undetected before production deploys.

2. Step-by-Step Professional Transformation Roadmap

To achieve enterprise readiness and excel during FYP defense, execute the following 7-phase structured transformation plan:

Phase 1: Enterprise Security & Cryptographic Session Hardening

Target Problem: src/lib/auth.ts uses Buffer.from(sessionData).toString('base64'). Any user can decode the cookie in DevTools, change role: 'STUDENT' to 'ADMIN', re-encode, and gain total platform control.

Step-by-Step Implementation:

1.1. Install jose or jsonwebtoken for cryptographically signed JWT sessions with HMAC SHA-256.
1.2. Store a cryptographically secure AUTH_SECRET (32+ bytes) in .env.
1.3. Sign session payloads with expiration (e.g. 7 days) and tamper detection (JWS).
1.4. Permanently remove backdoor passwords (Admin@123, Trainer@123, Student@123) from production logic.
1.5. Introduce password reset workflow with time-limited signed email tokens via Resend / SendGrid.
1.6. Create src/middleware.ts for edge-level route guarding on /dashboard/*, /learn/*, and API routes.

Phase 2: Database Migration to Production PostgreSQL & Schema Refinement

Target Problem: SQLite stores data in a single local disk file (prisma/dev.db). Under concurrent student traffic, SQLite locks the entire database on writes. Modern serverless platforms (Vercel, AWS ECS, Railway) recreate containers on deployment, wiping local database files.

Step-by-Step Implementation:

2.1. Switch Prisma provider in schema.prisma from provider = 'sqlite' to provider = 'postgresql'.
2.2. Provision a cloud PostgreSQL database instance (Neon Serverless Postgres, Supabase, or AWS RDS).
2.3. Initialize version-controlled database migrations via npx prisma migrate dev --name init_postgres.
2.4. Add composite database indexes: @@index([courseId]), @@index([userId]), @@index([slug]) for sub-millisecond query resolution.
2.5. Implement an AuditLog entity to track critical administrative mutations (role escalations, course takedowns, price modifications).

Phase 3: Real-Time Classroom Experience & Media Pipeline

Target Problem: Lessons currently rely on static YouTube embeds. Students can mark lessons completed instantly without watching, and instructors cannot upload proprietary videos or course resources.

Step-by-Step Implementation:

3.1. Integrate cloud asset storage (AWS S3 or Cloudinary) with pre-signed upload URLs for trainer video modules, lesson slides, and resource zip files.
3.2. Implement Video Heartbeat Tracking: dispatch lesson watch progress (e.g. every 10 seconds) to record exact timestamps and enforce an 80% watch threshold before completion toggle unlocks.
3.3. Add a dedicated Course Q&A; / Discussion Forum for each lesson where learners and faculty can engage in threaded discussions.
3.4. Implement Student Assignment Submissions: allow learners to upload homework/project files with a trainer grading interface and feedback notes.

Phase 4: Financial Transactions, Checkout & E-Commerce

Target Problem: enrollInCourseAction in src/app/actions/enrollment.ts instantly grants full course access for free, completely ignoring the course tuition fee configured in the database.

Step-by-Step Implementation:

4.1. Introduce financial schema models: Order, Transaction, and Invoice.

4.2. Integrate Stripe Checkout or PayFast SDK with server-side session generation.

4.3. Implement an idempotent Stripe webhook listener (/api/webhooks/stripe) to verify cryptographic signatures before activating course enrollment.

4.4. Build a Student Invoices & Purchase History page with downloadable tax-compliant PDF receipts.

4.5. Build a Trainer Revenue & Earnings Ledger showing course sales, platform commission deduction, and net faculty payout balance.

Phase 5: Automated Certificate Engine with Vector PDF & QR Code Verification

Target Problem: Certificates currently exist only as a database code (e.g. SN-2026-X89J21). While the verification page works, students cannot download a professional, high-resolution certificate for LinkedIn or print.

Step-by-Step Implementation:

5.1. Implement a server-side dynamic PDF Certificate Generation API route (/api/certificates/[code]/pdf).

5.2. Embed an authentic 2D QR Code on the certificate linking directly to https://skillnovaacademy.com/verify?code=....

5.3. Include academic metadata: Student Name, Course Title, Lead Instructor Signature, Issue Date, Final Grade / Score, and Official Academy Seal.

5.4. Add a one-click 'Add to LinkedIn Profile' button with pre-filled certification credentials.

Phase 6: Quality Assurance, Automated Testing & CI/CD DevOps

Target Problem: The repository has zero automated tests. Regressions can silently break core revenue or learning paths.

Step-by-Step Implementation:

6.1. Configure **Vitest** for server action and utility testing (auth verification, quiz scoring math, enrollment triggers).

6.2. Configure **Playwright** for End-to-End browser smoke testing (Student registration -> Course enrollment -> Lesson completion -> Certificate issue).

6.3. Add a multi-stage production Dockerfile with standalone Next.js output and non-root execution.

6.4. Setup GitHub Actions workflow (.github/workflows/ci.yml) running linting, TypeScript type-checking, Prisma schema validation, and test suites on every pull request.

Phase 7: Performance Optimization, Real-Time Notifications & SEO

Target Problem: Notifications are currently static database records fetched on page load. Dynamic course discovery needs rich metadata for search engines.

Step-by-Step Implementation:

7.1. Implement Server-Sent Events (SSE) or Pusher for real-time in-app notification toasts (e.g. trainer grades assignment -> student gets instant ping).

7.2. Implement dynamic src/app/sitemap.ts indexing all published courses, instructors, and categories for Google indexing.

7.3. Embed JSON-LD structured data (schema.org/Course, schema.org/EducationalOccupationalCredential) to generate Google Rich Course Snippets in search results.

3. Architectural Comparison: Academic vs. Commercial Enterprise LMS

The following matrix outlines the direct technological upgrades required across each functional tier:

Functional Tier	Current Academic State (FYP)	Target Commercial State	Engineering Effort
Session Management	Plain Base64 encoded cookie	Signed JWT with HMAC-SHA256 & rotating tokens	1 - 2 Days
Database Engine	Local SQLite (dev.db)	Cloud PostgreSQL with pooling & automated backups	2 - 3 Days
Payment Processing	None (Free bypass on all courses)	Stripe / PayFast webhooks + Invoicing ledger	3 - 4 Days
Video Hosting	Hardcoded YouTube iframes	AWS S3 / Cloudflare Stream with watch-time sync	3 - 4 Days
Certificates	Database code verification only	Downloadable Vector PDF with dynamic QR validation	1 - 2 Days
Automated Testing	Manual testing only	Vitest unit tests + Playwright E2E pipeline	3 - 4 Days
Containerization	Local dev server (npm run dev)	Multi-stage Dockerfile + Docker Compose	1 Day

4. Recommended Priority Milestones for FYP Defense

To maximize academic grading criteria and demonstrate commercial viability during evaluation:

• **Milestone 1 (Week 1 — Security & Integrity):** Replace Base64 session cookies with signed JWT tokens; add Next.js middleware.ts; purge fallback passwords.

• **Milestone 2 (Week 2 — Persistence & Scale):** Migrate Prisma from SQLite to PostgreSQL; establish versioned database migration scripts; add indexing.

• **Milestone 3 (Week 3 — Monetization & Receipts):** Add Stripe Checkout webhook integration; create Order and Invoice records; gate course content based on verified payment status.

• **Milestone 4 (Week 4 — Credentialing & Delivery):** Implement dynamic vector PDF certificate generation with live verification QR codes; enable student project file uploads.

• **Milestone 5 (Week 5 — DevOps & Defense Polish):** Add Vitest/Playwright test suites; multi-stage Docker build; deploy live staging on Vercel with Neon PostgreSQL.

Report Prepared By:
Antigravity Senior Architecture Agent
Google DeepMind Advanced Coding Team

Project Lead:
Murtaza Wazir
SkillNova Academy Lead Instructor

Target Defense:
BS Computer Science
Final Year Project (FYP)